

Blueprint for Autonomous Defensive Remediation: Multi-Agent Architecture for the PS3 Autonomy Track

Phase 1: System Architecture and Mathematical Logic

The foundational requirement of the PS3 Multi-Agent Autonomy track demands a paradigm shift away from traditional, single-shot Large Language Model (LLM) wrappers.¹ Systems relying on unconstrained conversational memory loops inevitably collapse under the weight of context degradation, non-deterministic routing, and hallucinated state transitions.² To construct a resilient, long-running, and fault-tolerant multi-agent system capable of asynchronous task orchestration, the architecture must abandon conversational looping in favor of a formal, mathematically rigorous graph-based execution model integrated with durable external state management.¹

The State Machine Engine: Formalizing Agent Orchestration

Multi-agent orchestration requires a definitive control flow mechanism to prevent race conditions, deadlocks, and corrupted state transitions during execution.⁵ The optimal architectural logic for coordinating specialized, asynchronous autonomous agents is the Colored Petri Net (CPN), which significantly extends the capabilities of standard Directed Acyclic Graphs (DAGs).³ While a standard DAG is highly effective for dictating predefined, sequential, or strictly parallel execution pipelines, it struggles to natively accommodate asynchronous, event-driven loop backtracks where an agent must deterministically re-evaluate a specific state subsequent to a tool failure.⁸

A Colored Petri Net model addresses these computational limitations by defining the multi-agent system as a bipartite graph composed of distinct places (representing system states) and transitions (representing the execution of agent actions).⁸ Within this framework, the multi-agent execution state can be formalized mathematically as a tuple. The finite set of places represents exact milestones, such as a pending reconnaissance phase, a ready-to-fire exploit payload, or a verified source code patch. The transitions define the strict boundaries of what an agent is permitted to execute, strictly gated by flow relations that represent data contract prerequisites. If an agent fails to fulfill the exact JSON schema required by the weight of the flow, the transition simply does not fire, deterministically halting the execution before it can corrupt the downstream workflow.⁹

By enforcing this mathematical structure, the central Orchestrator agent ceases to function as an unpredictable LLM generating arbitrary next steps. Instead, it operates strictly as a deterministic state transition engine.⁴ The reasoning capabilities of the underlying LLM are only invoked to evaluate the specific contextual conditions required to satisfy the transition, while

the physical routing logic itself remains hardcoded in Python.⁹ Frameworks that prioritize state machine architectures, such as LangGraph, excel in this environment by operating on a structured state that is passed from node to node, forcing the agent to repeatedly confront errors within a constrained feedback loop rather than silently ignoring them or hallucinating a bypass.⁴

Master State Persistence and the Redis-SQLite Hybrid Architecture

Operating within a 24-hour hackathon environment requires the system to be intrinsically crash-safe and capable of demonstrating absolute reliability during evaluation.¹ Ensuring this level of resilience mandates externalizing the "Master State" entirely outside of the LLM's ephemeral context window.¹ Relying on the LLM's context window for state persistence is a catastrophic anti-pattern that leads directly to token exhaustion, severe attention degradation, and total context collapse during long-running tasks.³

To solve this, the Master State must be managed via a highly optimized external database topology, specifically utilizing a Redis and SQLite hybrid persistence pattern.² This pattern cleanly divides operational state tracking from high-throughput asynchronous coordination.¹¹

SQLite, configured in Write-Ahead Logging (WAL) mode, serves as the durable, transactional checkpointer for the state machine.¹¹ Every time an agent successfully completes a transition within the Petri net, the updated state payload—strictly serialized as a JSON object—is atomically committed to the SQLite database.³ If the localhost server crashes mid-execution, the Orchestrator reads the most recent checkpoint directly from SQLite and resumes execution at the exact node in the graph where it failed, entirely bypassing the need to re-run prior, expensive LLM inferences.¹³

Concurrently, Redis handles the high-velocity, in-memory coordination required for real-time messaging between the Orchestrator and the distributed Worker agents.⁵ Redis Streams serves as the asynchronous event bus, ensuring that when the Orchestrator delegates multiple tasks simultaneously, the subsequent responses from the workers are sequenced deterministically.⁵ This semantic caching and pub/sub infrastructure prevents the systemic state corruption that occurs when multiple agents attempt to write to the state machine simultaneously.²

Persistence Layer	Architectural Function	Primary Objective	Data Structure
SQLite	Graph Checkpointing	Durable state recovery post-crash; deterministic	Relational tables storing serialized JSON

		resumption of the workflow.	state graphs.
Redis Streams	Event-Driven Bus	Sub-millisecond messaging between the Orchestrator and specialized Worker agents.	Append-only event logs with consumer group tracking.
Redis LangCache	Semantic Caching	Storing and retrieving previously generated LLM responses to reduce token latency.	Key-value pairs indexed by hashed tool signatures.

Asynchronous Fan-Out and Optimistic Concurrency Logic

To achieve true asynchronous orchestration—where tasks "fan out, complete on their own timeline, return to the planner" as mandated by the PS3 parameters—the system must implement a heavily decoupled Orchestrator-Worker pattern utilizing a robust message broker.¹ When the Orchestrator identifies parallelizable subtasks, such as initiating distinct reconnaissance scans across three different network ports simultaneously, it must never block the main execution thread.⁷

Instead, the Orchestrator publishes individual task payloads to a dedicated message queue. To prevent devastating race conditions and state-locking errors during the fan-in (aggregation) phase, the system implements Optimistic Concurrency Control (OCC) paired with deterministic UUID generation.⁶ The orchestration flow operates through strict segregation of duties.¹⁴

First, during task delegation, the Orchestrator generates a unique global Trace ID alongside a localized Task ID for each parallel subtask. It serializes the necessary operational context into a strict JSON payload and pushes this instruction set to a Redis-backed Celery or RQ queue.¹¹ The use of a data streaming technology like Redis allows the Orchestrator to use key-based partitioning strategies to distribute command messages effortlessly across available partitions.¹⁴

Second, during worker execution, specialized Worker agents operating in entirely isolated computing processes act as a consumer group.¹⁴ They pull tasks from the queue

independently, execute their LLM reasoning and localized tool calls, and manage their own ephemeral context windows without cross-contamination. The Kafka-like consumer rebalance protocols within Redis Streams assure that each worker agent maintains a similar workload, and in the event of a worker failure, the log can be replayed from a saved offset to guarantee execution.¹⁴

Finally, during the critical state synchronization phase (fan-in), the Worker agents are explicitly prohibited from directly overwriting the Master State in SQLite.⁶ Instead, they append their execution results back to a designated Redis Stream. The Orchestrator consumes this response stream, applying a deterministic, thread-safe reducer function to selectively merge the incoming results into the Master State.⁵ This append-only design paradigm completely eliminates write-write conflicts and guarantees that concurrent agent operations do not corrupt the foundational workflow.⁶

Webhook Ingress Architecture and ASGI Event Loop Design

The primary trigger for this autonomous pipeline is external event ingress; the system must react to the world autonomously rather than waiting for a user prompt.¹ This requires a highly decoupled HTTP listener capable of absorbing massive traffic spikes and cleanly dropping events into the queue architecture without blocking the event loop.¹

The optimal architectural design utilizes FastAPI, functioning as an Asynchronous Server Gateway Interface (ASGI) web server, configured purely as a high-speed reverse proxy for the Celery message queue.¹¹ The FastAPI ingress endpoint listens for incoming HTTP POST events, such as a webhook notification from a new staging deployment or a critical alert from a CVE RSS feed.¹

Immediately upon receipt, the payload is parsed through a strict Pydantic v2.11.7 schema definition to ensure absolute structural integrity.¹² If the payload lacks required fields, such as a target deployment URL or a valid authentication signature, the server rejects it with an HTTP 400 Bad Request in under ten milliseconds, protecting the downstream agents from processing malformed data.¹²

Valid payloads are instantly serialized and dispatched to the Celery queue via an asynchronous task trigger.¹¹ Crucially, the FastAPI server immediately returns an HTTP 202 Accepted response to the external caller, accompanied by the generated Trace ID, without waiting for the LLM or the Orchestrator to begin processing the event.¹¹ This absolute temporal decoupling ensures the ingress remains highly available. The background Celery worker subsequently picks up the task, initializes the Petri net state in the SQLite checkpointer, and wakes up the Orchestrator agent to initiate the complex, long-running workflow.¹¹

Phase 2: Step-by-Step Build Tasks (The 24-Hour Execution Plan)

Executing a production-grade, distributed multi-agent architecture within the rigid constraints of a 24-hour hackathon requires ruthless prioritization, aggressive compartmentalization of code, and a strict chronological build schedule.¹ The following blueprint breaks down the exact tasks required to construct the system from zero to a flawless final demonstration.

Task 1: Setting up the Ingress and Queue Infrastructure (Hours 0-4)

The primary objective of the first four hours is establishing the basic nervous system of the application. Before any LLM inference occurs, the system must successfully receive a webhook, enqueue the payload, and trigger a dummy worker process.¹¹

The engineering team begins by initializing a Python uvicorn server to host the FastAPI application. Simultaneously, a local Redis container is deployed via Docker to serve as the high-speed, in-memory broker. Celery is then configured to utilize this Redis instance as both its message broker and its result backend.¹¹

The developers construct the primary POST /webhook endpoint. This endpoint is wrapped in a Pydantic model that rigorously enforces the expected structure of the incoming webhook, guaranteeing that elements like the target_url and deployment_id are present and correctly typed.¹² The success criteria for this phase are strictly mechanical: a curl command sent to the localhost webhook must instantly return a 202 Accepted status, and the terminal monitoring the Celery worker must log the successful receipt and initialization of the task payload.¹¹

Task 2: Defining Strict JSON Data Contracts (Hours 4-8)

The next four hours are dedicated to eradicating the possibility of "Context Poisoning" and string-parsing errors by formalizing exactly what data is permitted to pass between the specialized agents.⁹ Without strict contracts, agents will inevitably pass unstructured natural language back and forth, leading to complete system collapse.³

The team defines a comprehensive library of Pydantic BaseModel classes for every agent's input and output expectations.¹² For example, the Recon_Output class must mathematically guarantee the inclusion of a list of discovered endpoints, detected vulnerabilities, and a network topology map.¹⁷ The Exploit_Output must contain the exact proof-of-concept code utilized, the standard output execution trace from the sandbox, and a strict Boolean indicating whether the vulnerability is actively exploitable.¹⁷

These Pydantic models are then converted into strict JSON Schemas, leveraging Draft 2020-12 specifications.¹⁷ The underlying LLM providers (e.g., OpenAI or Claude) are strictly configured to utilize these JSON Structured Outputs directly within their API calls.¹⁸ This static library of data classes serves as the unalterable canonical source of truth for all inter-agent message passing, ensuring that every handoff is structurally perfect.⁹

Task 3: Building the Orchestrator Logic (Hours 8-16)

With the communication protocols established, the team spends the next eight hours

constructing the deterministic state machine that will route data between the specialized agents.⁴ This orchestrator must entirely avoid relying on LLM conversational routing to make critical path decisions.¹⁰

Utilizing a framework designed for state machine orchestration, the developers define the explicit nodes of the graph: the ingress trigger, the reconnaissance agent, the exploitation agent, the patching agent, and crucially, the deterministic verifier agent.¹⁰ The conditional edges—the transitions between these nodes—are programmed using pure, deterministic Python if/else logic.¹⁰ For instance, the edge routing execution from the Exploit Agent to the Patch Agent is only traversed if the parsed Exploit_Output JSON explicitly states `Is_Vulnerable == True`. If the payload indicates False, the system routes directly to an End_Workflow node, preventing unnecessary downstream execution.⁹

During this phase, the SQLite checkpointing layer is wired directly into the graph's execution loop. The developers configure the graph to serialize and commit the state dictionary to the SQLite database after every single node execution.⁷ The deliverable for this phase is a fully functioning graph where dummy nodes pass mocked JSON data to one another, successfully progressing from the start state to the end state, proving the underlying mathematical topology is sound.¹⁰

Task 4: Implementing the Isolated Tool Execution Layer (Hours 16-24)

The final eight hours are dedicated to connecting the actual Large Language Models and engineering the tool execution layer so that the system can enact real-world side effects safely within the localhost environment.¹

The developers wrap the necessary tool definitions—such as executing a Python script, querying a CVE database, or initiating a Git commit—using standard JSON schemas that the LLMs can reliably parse and invoke.¹⁹ Because the project involves an autonomous red-team system generating potentially dangerous exploit payloads, the execution must be strictly confined to a localhost sandbox to prevent the host machine from being corrupted.¹

To achieve this, the team implements an ephemeral execution environment utilizing Pyodide (which compiles Python to WebAssembly) or a highly isolated Docker container mapped to a restricted, read-only volume with explicitly disabled network bridging.⁹ The actual LLM API calls are injected into the DAG nodes, providing them with the tool schemas and rigid system prompts.¹⁹ The final objective is an end-to-end run where a webhook triggers the Orchestrator, the LLMs generate the required structured payloads, the exploitation tools execute securely within the sandbox, and a verifiable source code patch is synthesized.¹

Phase 3: Surviving "Vibe Coding" and AI Pitfalls

The most frequent and critical failure mode in multi-agent hackathon projects is an over-reliance on probabilistic, unbounded LLM execution loops.⁹ When developers treat agents as magical black boxes rather than probabilistic engines, the agents hallucinate successful tool

calls, drown in excessively large context windows, and inevitably become trapped in infinite retry spirals.⁹ To survive the live evaluation panel and produce a product that reliably executes, the system must implement hardcore "Fail-Closed" circuit breakers and rigorous deterministic validation logic.⁹

Eradicating Hallucinated Side-Effects via the Verifier Agent

Large Language Models possess a well-documented failure mode wherein they will confidently generate text asserting, "I have successfully exploited the vulnerability and patched the server," without ever having generated the correct JSON required to actually invoke the corresponding tool.⁹ In a distributed multi-agent pipeline, if an agent hallucinates a successful action, the error propagates downstream as trusted input, silently corrupting the entire workflow.³

To structurally prevent this phenomenon, the architecture must implement a specialized "Verifier Agent" pattern.²⁰ The Verifier Agent is not a traditional generative LLM; it is an independent, deterministic validation node in the state machine whose sole responsibility is to cryptographically or procedurally confirm that a tool actually produced a verifiable side-effect before allowing the graph to transition to the next state.⁹

If the Exploiter Agent claims to have read a sensitive file (such as a `/etc/shadow` equivalent located within the isolated sandbox), the Verifier Agent does not evaluate the Exploiter's natural language reasoning.⁹ Instead, the Verifier strictly evaluates the JSON output generated directly by the isolated code execution tool. If the `stdout` field of the tool's response payload does not contain the exact expected cryptographic hash, the explicit integer result, or the specific string pattern associated with the target file, the Verifier unequivocally rejects the state transition.⁹

This procedural cross-validation forces the system to compare the `Action_Requested` against the actual `System_State_Change`.²⁵ The Verifier acts as an adversarial entity, mathematically grading the output against a required schema. If the validation fails, the Verifier generates a highly structured error payload containing the precise error trace and returns it to the Orchestrator, triggering a targeted re-generation attempt or gracefully halting the pipeline.⁹

Context Poisoning and Attention Collapse Prevention

When a Reconnaissance agent utilizes a web scraper to pull down an entire HTML Document Object Model (DOM) or a massive block of web server logs, passing this raw, uncompressed string directly into the context window of the downstream Exploiter agent will inevitably cause "attention collapse".³ The LLM loses its ability to focus on its primary directive, becomes distracted by irrelevant noise within the logs, and begins hallucinating false vulnerabilities.⁹

To serve as a circuit breaker against context poisoning, agents must be explicitly forbidden from passing raw data strings to one another.⁹ The architecture mandates the insertion of an intermediate, purely deterministic parsing layer.⁹

When the Recon agent executes a web search or scraping tool, the raw resulting data is first

pushed through a deterministic Python algorithm—such as BeautifulSoup for HTML or a Regex parser for logs—to extract only the functional, high-value artifacts (e.g., target URLs, exposed input fields, exact API endpoints).⁹ These highly distilled artifacts are then serialized into the predefined Pydantic schema.¹² The downstream Exploiter agent receives *only* the highly compressed, strictly typed JSON object, completely isolated from the raw HTML or log data.⁹ This guarantees that the agent's attention mechanism remains entirely focused on exploitation logic rather than parsing noisy data.

Deterministic Loop-Breakers and Fail-Closed Middleware

If an agent writes syntactically invalid Python code, standard agentic frameworks will blindly pass the raw syntax error back to the agent and prompt it to correct the mistake.⁴ This lack of supervision frequently results in an infinite "death spiral," where the agent repeatedly hallucinates the same incorrect fix, draining API credits, exceeding hackathon time limits, and causing the demonstration to crash catastrophically.⁴

To prevent infinite tool-retry loops, the execution cycle must be wrapped in hardcoded Python middleware that enforces a strict fail-closed architecture.²³ This requires migrating from interpretation-based execution to a "Compiled AI" validation pipeline.²⁵

Loop-Breaker Mechanism	Implementation Details	Operational Impact
Absolute Iteration Limits	A strict integer counter is embedded directly into the graph state. If <code>retry_count > 3</code> , the execution engine raises a deterministic exception. ⁹	Prevents the system from infinitely looping on unsolvable problems, ensuring resources are preserved. ⁹
Deterministic Syntax Validation	Generated code is passed through an Abstract Syntax Tree (AST) parser (e.g., <code>ast.parse</code>) and a linter (e.g., <code>ruff</code>) <i>before</i> it is ever executed or sent back to the LLM. ⁹	If the code is unparseable, it fails a fast, CPU-bound check in milliseconds, bypassing expensive LLM inference entirely. ⁹
Signature Hashing	The middleware intercepts the tool call and creates a cryptographic hash of the exact tool signature (the tool name combined with	If the LLM attempts to execute the exact same failed signature twice, the circuit breaker deterministically blocks it,

	its specific arguments). ²⁶	forcing the agent to formulate a novel approach. ²⁶
Fail-Closed Locking	If network connectivity drops or the validation logic crashes, the middleware defaults to a closed state, physically preventing the tool from firing. ²³	Guarantees that the agent cannot execute potentially dangerous commands if the safety telemetry goes offline. ²³

Phase 4: Omium SDK Observability Integration

To secure the highly coveted bonus points during the evaluation panel, the submitted product must provide irrefutable, verifiable proof of its autonomous execution.¹ OpenTelemetry (OTLP) represents the definitive industry standard for distributed systems observability, and integrating the Omium SDK ensures that every granular action—from the initial webhook trigger to the asynchronous dispatch, LLM reasoning steps, and final tool executions—is deterministically linked within a causal graph.²⁷

W3C Trace Context Propagation Across Asynchronous Boundaries

In distributed, asynchronous multi-agent systems, observability traces frequently break when execution crosses process boundaries (for instance, when a payload moves from the synchronous FastAPI listener into the asynchronous Celery queue).²⁸ To maintain the unbroken causal chain required by the judges to prove end-to-end autonomy, the system must manually propagate the W3C Trace Context headers across these boundaries.²⁸

The process begins during the Ingress Generation phase. When the external webhook strikes the FastAPI endpoint, the Omium SDK middleware—configured to export telemetry to ingest.monium.yandex.cloud:443—automatically generates a unique 16-byte Trace ID (e.g., 4bf92f3577b34da6a3ce929d0e0e4736) and an initial 8-byte root Span ID.²⁷

To survive the jump to the message queue, these identifiers are serialized. The Trace ID and Span ID are formatted into a standard W3C traceparent string (e.g., 00-{trace_id}-{span_id}-01).²⁸ This exact traceparent string is explicitly injected into the metadata payload of the Celery task broker alongside the actual execution instructions.²⁸

Upon initialization, the Celery worker extracts the traceparent string from the task metadata. The Omium SDK instance operating on the isolated worker node is manually initialized using this extracted context.³⁰ Consequently, all subsequent operations executed by the worker—such as database queries, tool invocations, or LLM inferences—are automatically appended as children to the original webhook trace, successfully bridging the asynchronous gap and preserving the integrity of the trace.²⁸

Causal Threading and Sub-Graph Span Alignment

To prove the depth of the system's "thought process" to the judging panel, the trace architecture must clearly delineate between the orchestrator's high-level decision-making logic and the worker's low-level tool execution.¹ This requires mapping the execution to a strict hierarchical tree of spans.²⁸

By classifying the "Span Kind" according to OpenTelemetry GenAI Semantic Conventions, the system categorizes outgoing requests to LLM APIs as CLIENT spans, while internal reasoning, agent turns, and deterministic tool executions are recorded as INTERNAL spans.²⁸

By utilizing Omium's intuitive context managers (e.g., with `omium_trace.trace_operation("Agent_Decision")` as span:), developers can attach critical metadata attributes directly to the spans to prove autonomy.³² Vital metadata that must be appended includes exact token counts (`llm.prompt_tokens` and `llm.completion_tokens`), the precise model parameters, and the `agent.decision_rationale` (the internal thought process extracted from the JSON schema).³² Furthermore, the `tool.execution_status` must be logged to cryptographically prove whether the isolated sandbox execution succeeded or failed.³¹

When the judges open the Omium dashboard during the live evaluation, they will visualize a hierarchical, visually distinct Directed Acyclic Graph of spans that maps perfectly to the execution timeline shown in the demo video.¹ This transparent causal threading definitively proves that the system achieved true multi-agent autonomy without relying on scripted prompts or hidden human intervention.¹

Phase 5: The Exact Build (The Autonomous Red-Team Engine)

The culmination of this architecture is the deployment of an autonomous penetration testing and remediation pipeline.²¹ Built entirely to operate within the strict confines of an isolated localhost sandbox, this system proactively targets a designated staging deployment, discovers vulnerabilities, generates and fires live exploit payloads to verify their existence, and subsequently authors the GitHub Pull Request required to patch the flaw.¹

The Autonomous Red-Team Workflow Sequence

The system operates across three specialized agent domains, orchestrated through the state machine.

1. **Trigger:** A staging deployment webhook (simulated via a curl command) notifies the FastAPI ingress, initiating the Omium trace and dropping the payload into the Celery queue.
2. **Reconnaissance (Agent 1):** The Recon agent utilizes search tools and Nmap integrations to scan the target localhost port, fingerprint the running service, and cross-reference a local CVE database to identify potential attack vectors.²¹

3. **Exploitation (Agent 2):** Based on the reconnaissance data, the Exploiter agent generates a highly targeted Python payload designed to exploit the discovered vulnerability (e.g., a path traversal or injection flaw). It executes this payload within the isolated Pyodide or Docker sandbox. The payload's objective is to read a hidden flag file (e.g., `/tmp/secret_flag.txt`).²¹
4. **Verification:** The deterministic Verifier node intercepts the result, checking if the tool's standard output explicitly contains the secret flag, proving the vulnerability is real and not hallucinated.⁹
5. **Patching (Agent 3):** Upon verification, the Patcher agent clones the target repository to a temporary local directory, alters the Abstract Syntax Tree (AST) of the vulnerable code, commits the change, and generates a structured GitHub Pull Request containing the exploit proof.¹⁷

JSON Schema Definitions for Deterministic Inter-Agent Handoffs

To facilitate seamless handoffs without context poisoning, the agents communicate exclusively using strict JSON Schema Draft 2020-12 representations, enforced dynamically via Pydantic.¹⁷

Agent 1: Reconnaissance Output Schema

The output from the Reconnaissance agent must strictly catalog the discovered attack surface without including raw HTTP responses.

JSON

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "ReconOutput",
  "type": "object",
  "properties": {
    "target_url": { "type": "string", "format": "uri" },
    "detected_framework": { "type": "string" },
    "vulnerable_endpoints": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "path": { "type": "string" },
          "method": { "type": "string", "enum": [] },
          "injection_vector": { "type": "string" }
        }
      }
    }
  },
}
```

```

    "required": ["path", "method", "injection_vector"]
  }
},
"required": ["target_url", "detected_framework", "vulnerable_endpoints"]
}

```

Agent 2: Exploiter Output Schema

The Exploiter agent must return the exact proof-of-concept code and the results of the sandbox execution.

JSON

```

{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "ExploitOutput",
  "type": "object",
  "properties": {
    "vulnerability_confirmed": { "type": "boolean" },
    "exploit_payload_used": { "type": "string" },
    "sandbox_stdout": { "type": "string" },
    "extracted_secret": { "type": "string" }
  },
  "required": ["vulnerability_confirmed", "exploit_payload_used", "sandbox_stdout"]
}

```

Agent 3: Patcher Output Schema

The Patcher agent synthesizes the remediation logic into a diff format ready for version control integration.

JSON

```

{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "PatchOutput",
  "type": "object",

```

```

"properties": {
  "file_modified": { "type": "string" },
  "unified_diff": { "type": "string" },
  "pull_request_title": { "type": "string" },
  "pull_request_body": { "type": "string" },
  "confidence_score": { "type": "number", "minimum": 0, "maximum": 1 }
},
"required": ["file_modified", "unified_diff", "pull_request_title", "pull_request_body"]
}

```

Isolated Python Tool Execution Definitions

To execute the exploit payload securely, the Exploiter Agent must invoke a strictly defined Python tool that acts as the absolute boundary between the Orchestrator and the sandbox.²² This design ensures that malicious or unstable code generated by the LLM can never escape and compromise the host machine.⁹

The tool is defined using a Pydantic model (ExecuteSandboxPayload) that expects a string of Python code and a timeout integer.¹⁹ When invoked, the tool writes the LLM-generated payload to an ephemeral file (/tmp/payload.py). It then utilizes the Python subprocess module to spin up a highly restricted Docker container (python:3.11-slim).²² The container is launched with the `--rm` flag to ensure immediate destruction after execution, and it mounts the payload file as read-only (:ro).⁹

To prevent infinite loops within the generated code from locking the system, the `subprocess.run` command is hardcoded with a `timeout_seconds` parameter.⁹ If the exploit takes too long to execute, the subprocess raises a `TimeoutExpired` exception, the container is forcibly killed, and the circuit breaker returns a deterministic error to the Orchestrator, effectively halting the death spiral before it impacts the broader system.⁹

GitHub Pull Request Generation and AST Modification Architecture

The final requirement is the autonomous generation of a patch. Generating raw code as a string and attempting to overwrite a file is highly error-prone, as LLMs frequently introduce markdown formatting or omit required imports.⁹ To solve this, the Patcher Agent utilizes Abstract Syntax Tree (AST) modification.²⁵

The patching architecture operates through a strict sequence of Git and Python tool invocations. First, a tool clones the target repository into an isolated directory and checks out a new branch uniquely named using the Trace ID.³⁶ The Patcher agent is then instructed to generate the fix not as raw text, but as a Python script utilizing the `ast` module to programmatically rewrite the vulnerable node in the target file.²⁵

Once the AST script is executed and the file is modified, the system invokes a deterministic unit-testing node. The unit tests verify that the original exploit payload—which succeeded

during the Exploiter phase—now fails, proving the patch is effective.⁹ Finally, the system executes git add, git commit (utilizing the generated pull_request_title from the JSON schema), and git push, concluding the workflow by generating a PR that contains the exact standard output from the sandbox as irrefutable proof of both the vulnerability and the remediation.⁹

By treating the multi-agent system as a formal Petri net, utilizing SQLite and Redis for robust state and concurrency management, enforcing strictly typed JSON data contracts, and implementing deterministic Fail-Closed circuit breakers, the architecture completely eradicates the unpredictable nature of traditional LLM wrappers.¹ Integrating the Omium SDK via W3C Trace Context propagation transforms this robust backend into a highly observable, verifiable workflow, ensuring the system not only completes its objective but provides mathematical proof of its autonomy.²⁸ The resulting Autonomous Red-Team Engine represents a hardened, production-ready implementation of defensive cybersecurity automation that directly fulfills and exceeds every rigorous constraint of the PS3 evaluation rubric.

Works cited

1. 3.pdf
2. AI Agent Architecture Patterns: Single & Multi-Agent Systems - Redis, accessed May 15, 2026, <https://redis.io/blog/ai-agent-architecture-patterns/>
3. Multi-Agent AI Systems: Architecture & Failure Modes | Augment Code, accessed May 15, 2026, <https://www.augmentcode.com/guides/multi-agent-ai-systems>
4. Top 5 Open-Source Agentic AI Frameworks in 2026 - AIMultiple, accessed May 15, 2026, <https://aimultiple.com/agentic-frameworks>
5. AI agent orchestration for production systems - Redis, accessed May 15, 2026, <https://redis.io/blog/ai-agent-orchestration/>
6. Avoiding race conditions for advanced agents - Pega Documentation, accessed May 15, 2026, <https://docs.pega.com/bundle/platform/page/platform/background-processing/agents-avoid-race-conditions.html>
7. Building Multi-Agent AI Systems: Architecture Patterns and Best Practices - DEV Community, accessed May 15, 2026, https://dev.to/matt_frank_usa/building-multi-agent-ai-systems-architecture-patterns-and-best-practices-5cf
8. AI Agent Orchestration Patterns - Azure Architecture Center ..., accessed May 15, 2026, <https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/ai-agent-design-patterns>
9. How to Build a Structured AI Coding Workflow with Deterministic and Agentic Nodes, accessed May 15, 2026, <https://www.mindstudio.ai/blog/structured-ai-coding-workflow-deterministic-agentic-nodes>
10. Stop Asking LLMs to Be Deterministic - DEV Community, accessed May 15, 2026, <https://dev.to/demianbrecht/stop-asking-llms-to-be-deterministic-e32>
11. What Are Multi-Agent AI Systems - Runpod, accessed May 15, 2026,

- <https://www.runpod.io/articles/guides/what-are-multi-agent-ai-systems>
12. BjornMelin/dev-pro-agents: Advanced multi-agent orchestration framework built with LangGraph - Coordinate specialized AI agents for autonomous development, research, testing, and documentation workflows with intelligent task routing and real-time collaboration · GitHub, accessed May 15, 2026, <https://github.com/BjornMelin/dev-pro-agents>
 13. Multi-Agent Patterns | Pydantic Docs, accessed May 15, 2026, <https://pydantic.dev/docs/ai/guides/multi-agent-applications/>
 14. Four Design Patterns for Event-Driven, Multi-Agent Systems - Confluent, accessed May 15, 2026, <https://www.confluent.io/blog/event-driven-multi-agent-systems/>
 15. Hands-on Tutorial: Building an AI Agent with human-in-the-loop control - SAP Community, accessed May 15, 2026, <https://community.sap.com/t5/artificial-intelligence-blogs-posts/hands-on-tutorial-building-an-ai-agent-with-human-in-the-loop-control/ba-p/14050267>
 16. Building async processing pipelines with FastAPI and Celery on Upsun, accessed May 15, 2026, <https://developer.upsun.com/posts/tutorials/building-async-processing-pipelines-with-fastapi-and-celery-on-upsun>
 17. A Systematic Study of LLM-Based Architectures for Automated Patching - arXiv, accessed May 15, 2026, <https://arxiv.org/html/2603.01257v1>
 18. Agent orchestration - OpenAI Agents SDK, accessed May 15, 2026, https://openai.github.io/openai-agents-python/multi_agent/
 19. The Roadmap to Mastering Tool Calling in AI Agents - Machine Learning Mastery, accessed May 15, 2026, <https://machinelearningmastery.com/the-roadmap-to-mastering-tool-calling-in-ai-agents/>
 20. RIVA: Leveraging LLM Agents for Reliable Configuration Drift Detection - arXiv, accessed May 15, 2026, <https://arxiv.org/html/2603.02345v1>
 21. VulnBot: Autonomous Penetration Testing for A Multi-Agent Collaborative Framework - arXiv, accessed May 15, 2026, <https://arxiv.org/html/2501.13411v1>
 22. GitHub - langchain-ai/langchain-sandbox: Safely run untrusted Python code using Pyodide and Deno, accessed May 15, 2026, <https://github.com/langchain-ai/langchain-sandbox>
 23. I built a "Fail-Closed" Circuit Breaker for my Agent because prompts weren't enough to stop hallucinations. Open sourcing it today. (Python) - Reddit, accessed May 15, 2026, https://www.reddit.com/r/LocalLLaMA/comments/1q64zgt/i_built_a_failclosed_circuit_breaker_for_my_agent/
 24. Verification-Aware Planning for Multi-Agent Systems - arXiv, accessed May 15, 2026, <https://arxiv.org/html/2510.17109v1>
 25. Compiled AI: Deterministic Code Generation for LLM-Based Workflow Automation - arXiv, accessed May 15, 2026, <https://arxiv.org/html/2604.05150v1>
 26. Advice wanted: designing robust LLM inference loops with tools : r/LocalLLaMA - Reddit, accessed May 15, 2026,

- https://www.reddit.com/r/LocalLLaMA/comments/1qpp393/advice_wanted_designing_robust_llm_inference/
27. Monium Traces | Yandex Cloud - Documentation, accessed May 15, 2026, <https://yandex.cloud/en/docs/monium/traces/>
 28. AG2 OpenTelemetry Tracing: Full Observability for Multi-Agent Systems, accessed May 15, 2026, <https://docs.ag2.ai/latest/docs/blog/2026/02/08/AG2-OpenTelemetry-Tracing/>
 29. Tracing - OpenAI Agents SDK, accessed May 15, 2026, <https://openai.github.io/openai-agents-python/tracing/>
 30. Django tracing with OpenTelemetry - Adin Hodovic, accessed May 15, 2026, <https://hodovi.cc/blog/django-tracing-with-opentelemetry/>
 31. AI Agent Debugging & Tracing | End-to-End Visibility - Noveum.ai, accessed May 15, 2026, <https://noveum.ai/en/solutions/debugging>
 32. Noveum Trace SDK - GitHub, accessed May 15, 2026, <https://github.com/Noveum/noveum-trace>
 33. LLM Tracing & AI Tracing for Agents | MLflow AI Platform, accessed May 15, 2026, <https://mlflow.org/llm-tracing>
 34. PentestAgent Enables Autonomous Penetration Testing For Security Teams, accessed May 15, 2026, <https://letsdatascience.com/news/pentestagent-enables-autonomous-penetration-testing-for-secu-65b78085>
 35. AI Penetration Testing: Complete Guide to Autonomous Security Testing | VibeEval, accessed May 15, 2026, <https://vibe-eval.com/ai-pentest/ai-penetration-testing-guide/>
 36. Meet the Workflows: Fault Investigation | GitHub Agentic Workflows, accessed May 15, 2026, <https://github.github.com/gh-aw/blog/2026-01-13-meet-the-workflows-quality-hygiene/>